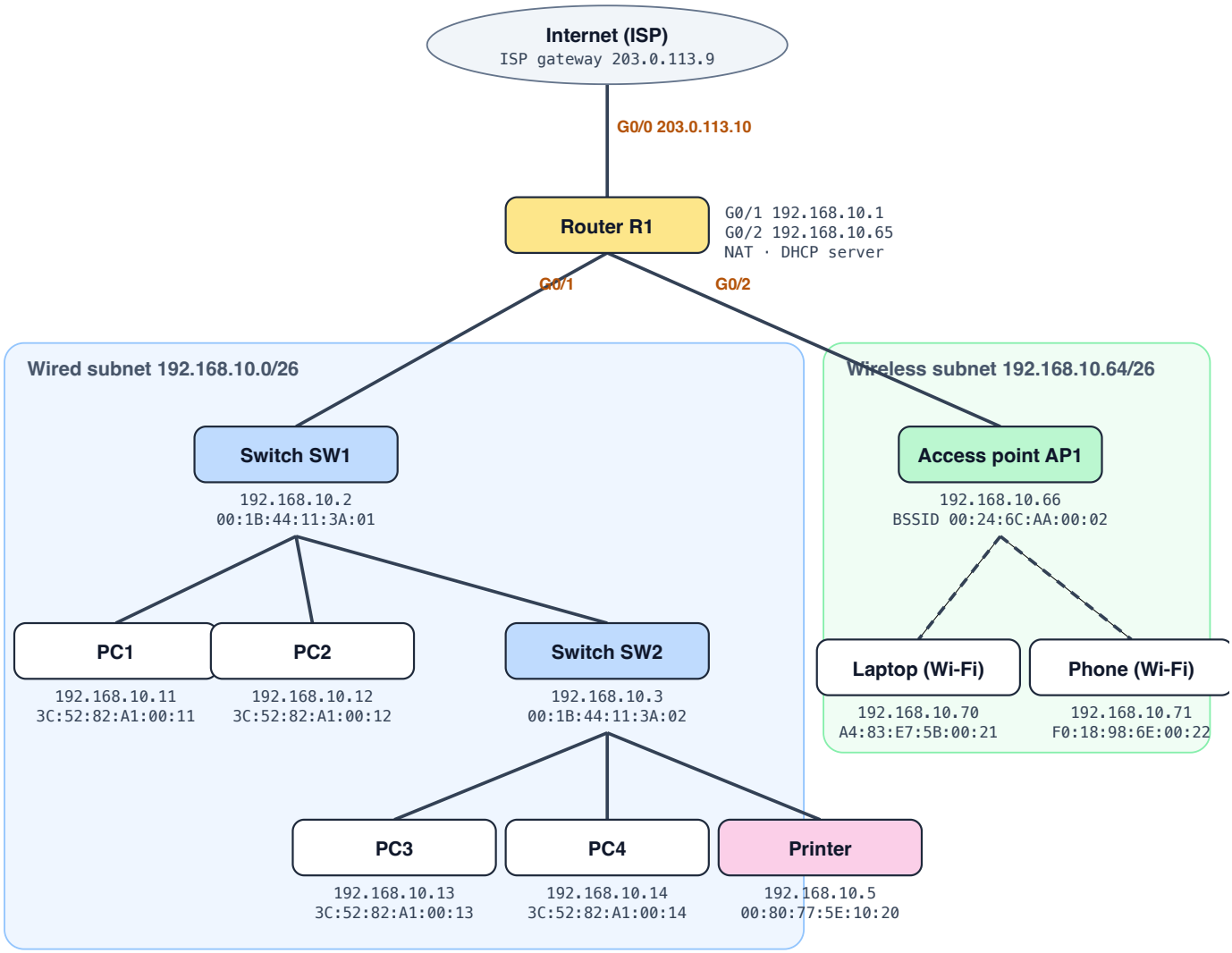
# Company Floor Network

LAN design · data link layer · IP addressing · routing and NAT

Chimaroke Amadi · GoMyCode

For this checkpoint I designed a small network for one company floor. It has one router with internet access, two switches, four employee PCs, a printer, and a Wi-Fi access point with two wireless devices. The report follows a frame and a packet through the network and explains what each layer does with them. All the addresses come from one data file, and a small checking script (`check.js`) confirms that the subnets don't overlap, every MAC and IP address is unique, every device is in the right subnet, and the DHCP pools don't clash with the fixed addresses.

## Network diagram




The router connects to the internet on **G0/0**. Its **G0/1** port goes to switch SW1, which connects PC1, PC2 and switch SW2. SW2 connects PC3, PC4 and the printer. The router's **G0/2** port goes to the access point, which serves the laptop and the phone over Wi-Fi (dashed lines). The wired and wireless devices are in two different subnets, so the router sits between them as well as between the LAN and the internet.

# Part 1: MAC Addresses and Frame Handling

## 1.1 MAC addresses

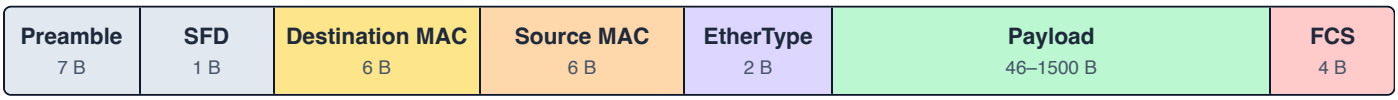
A MAC address is the 48-bit (6-byte) hardware address of a network interface, written as six pairs of hex digits. The first 3 bytes are the **OUI** (Organizationally Unique Identifier, which says who made the card), and the last 3 bytes identify the card itself. MAC addresses are used to deliver frames *within one local network*. Every interface has its own MAC, so the router has three (one per port) and the access point has two (its wired port, and its radio, whose MAC is the **BSSID** of the Wi-Fi network). These are example addresses I assigned for the design.

| Device | Interface | MAC address | OUI (maker) | Device part |
|---|---|---|---|---|
| Router R1 | G0/0 (WAN, to ISP) | `00:1A:2B:3C:4D:01` | `00:1A:2B` | `3C:4D:01` |
| Router R1 | G0/1 (wired LAN) | `00:1A:2B:3C:4D:02` | `00:1A:2B` | `3C:4D:02` |
| Router R1 | G0/2 (wireless LAN) | `00:1A:2B:3C:4D:03` | `00:1A:2B` | `3C:4D:03` |
| Switch SW1 | Management | `00:1B:44:11:3A:01` | `00:1B:44` | `11:3A:01` |
| Switch SW2 | Management | `00:1B:44:11:3A:02` | `00:1B:44` | `11:3A:02` |
| Printer | Ethernet | `00:80:77:5E:10:20` | `00:80:77` | `5E:10:20` |
| PC1 | Ethernet | `3C:52:82:A1:00:11` | `3C:52:82` | `A1:00:11` |
| PC2 | Ethernet | `3C:52:82:A1:00:12` | `3C:52:82` | `A1:00:12` |
| PC3 | Ethernet | `3C:52:82:A1:00:13` | `3C:52:82` | `A1:00:13` |
| PC4 | Ethernet | `3C:52:82:A1:00:14` | `3C:52:82` | `A1:00:14` |
| Access point AP1 | Ethernet (uplink + management) | `00:24:6C:AA:00:01` | `00:24:6C` | `AA:00:01` |
| Access point AP1 | Wi-Fi radio (BSSID) | `00:24:6C:AA:00:02` | `00:24:6C` | `AA:00:02` |
| Laptop (wireless 1) | Wi-Fi | `A4:83:E7:5B:00:21` | `A4:83:E7` | `5B:00:21` |
| Phone (wireless 2) | Wi-Fi | `F0:18:98:6E:00:22` | `F0:18:98` | `6E:00:22` |

All of them are **unicast** addresses (the lowest bit of the first byte is 0). The special address `FF:FF:FF:FF:FF:FF` is the **broadcast** address: a frame sent to it is delivered to every device in the same network. ARP and DHCP use it.

## 1.2 A frame from PC1 to the printer

PC1 (192.168.10.11) and the printer (192.168.10.5) are in the same subnet, so the data doesn't go through the router. It travels as an Ethernet frame through SW1 and SW2. Before sending, PC1 needs the printer's MAC address. If it isn't in its ARP cache, PC1 sends an **ARP request**, a broadcast frame (destination `FF:FF:FF:FF:FF:FF`, EtherType `0x0806`) asking "who has 192.168.10.5? Tell 192.168.10.11". Only the printer answers, with a unicast **ARP reply**: "192.168.10.5 is at 00:80:77:5E:10:20". Then PC1 sends the print job in an Ethernet II frame:



| Field | Size | Value in this frame | Purpose |
|---|---|---|---|
| **Preamble** | 7 B | `10101010 × 7` | Lets the receiver synchronise its clock |
| **SFD** | 1 B | `10101011` | Start Frame Delimiter: "the frame starts now" |
| **Destination MAC** | 6 B | `00:80:77:5E:10:20` | The printer's NIC (learned with ARP) |
| **Source MAC** | 6 B | `3C:52:82:A1:00:11` | PC1's NIC |
| **EtherType** | 2 B | `0x0800` | The payload is an IPv4 packet (0x0806 would be ARP) |
| **Payload** | 46–1500 B | `IPv4 packet` | IP header (192.168.10.11 → 192.168.10.5) + TCP header (port 9100) + print data |
| **FCS** | 4 B | `CRC-32` | Frame Check Sequence: the receiver recalculates it to detect damaged frames |

The payload is the IP packet. Its header has source 192.168.10.11, destination 192.168.10.5, TTL 64 and protocol 6 (TCP). Inside it is a TCP segment to port **9100** (raw printing; IPP on port 631 would also work) carrying the document. If the job is bigger than 1500 bytes, it's split into several frames. Very small payloads are padded up to 46 bytes, because an Ethernet frame must be at least 64 bytes for collision detection to work.

**How the switches deliver it:** a switch learns which MAC address is on which port by reading the *source* MAC of every frame it receives, and stores this in its **MAC address table** (CAM table). For the *destination* MAC, it forwards the frame only out of the port where that MAC was learned. If it doesn't know the MAC yet, it floods the frame out of every port except the one it came in on.

| **SW1 MAC address table** | |
|---|---|
| **MAC** | **Port** |
| `3C:52:82:A1:00:11` (PC1) | Fa0/1 |
| `3C:52:82:A1:00:12` (PC2) | Fa0/2 |
| `00:80:77:5E:10:20` (Printer) | Gi0/2 (to SW2) |
| `00:1A:2B:3C:4D:02` (Router) | Gi0/1 (to R1) |

| **SW2 MAC address table** | |
|---|---|
| **MAC** | **Port** |
| `3C:52:82:A1:00:11` (PC1) | Gi0/1 (to SW1) |
| `3C:52:82:A1:00:13` (PC3) | Fa0/1 |
| `3C:52:82:A1:00:14` (PC4) | Fa0/2 |
| `00:80:77:5E:10:20` (Printer) | Fa0/3 |

So the frame goes PC1 → SW1 (out Gi0/2) → SW2 (out Fa0/3) → printer. The switches **don't change the frame**: the MAC addresses, the IP packet and the TTL stay exactly the same, because switches work at layer 2. The printer checks the FCS, sees its own MAC as the destination, reads EtherType 0x0800 and passes the packet up to IP and then TCP.

### 1.3 Avoiding and handling collisions

A **collision** happens when two devices transmit on the same shared medium at the same time and their signals get mixed up.

**Wired Ethernet: CSMA/CD, and why switches make it unnecessary.** Old Ethernet used hubs or one shared cable, where all devices were in one *collision domain*. It used **CSMA/CD** (Carrier Sense Multiple Access with Collision Detection). A device listens and only transmits when the cable is quiet. It keeps listening while it transmits, and if it detects a collision it stops, sends a **jam signal** so everyone knows, and waits a random time before trying again (**binary exponential backoff**: after the n-th collision it waits a random number of slot times between 0 and $2^n$ – 1, giving up after 16 attempts). In this design the PCs and the printer are connected to

**switches in full-duplex mode**. Each switch port is its own collision domain with only one device on it, and full duplex uses separate wire pairs for sending and receiving, so **collisions can't happen** and CSMA/CD is effectively switched off. If two PCs send to the printer at once, the switch **buffers** the frames in a queue instead of letting them collide. All the wired devices are still in one *broadcast domain*, which the router limits, so broadcasts don't reach the Wi-Fi subnet.

**Wi-Fi: CSMA/CA.** Wireless is different. All devices share the same radio channel, and a device can't listen while it's transmitting (its own signal is much stronger), so it can't *detect* collisions. Instead, 802.11 tries to **avoid** them with **CSMA/CA** (Collision Avoidance):

1. **Listen first:** the device checks that the channel is free (physical carrier sense). It also checks the **NAV** (Network Allocation Vector), a timer set from the "duration" field of frames it overhears, which says how long the channel is already booked (virtual carrier sense).
2. **Wait DIFS plus a random backoff:** even when the channel is free, it waits a fixed gap (DIFS) and then a random number of slots from its *contention window*. The countdown pauses whenever someone else transmits. Because the laptop and the phone almost never pick the same number, they rarely start at the same moment.
3. **Transmit, then wait for an ACK:** the receiver (here the access point) checks the frame and sends an **ACK** after a short gap (SIFS). If there's no ACK, the sender assumes a collision or interference, **doubles its contention window** and tries again.
4. **RTS/CTS for hidden nodes (optional):** if the laptop and phone are far apart and can't hear each other, but both can hear the AP, they could collide at the AP without knowing it. With RTS/CTS the sender first sends a short *Request To Send*, and the AP answers *Clear To Send*. Everyone who hears the CTS sets their NAV and stays quiet.

> **In short:** wired Ethernet with switches has no collisions (each full-duplex port is its own collision domain, and CSMA/CD was only needed for shared, half-duplex links), while Wi-Fi shares the air, so it uses CSMA/CA with random backoff and ACKs to *avoid* collisions it can't detect.

## Part 2: IP Addressing and Subnetting

### 2.1 Choosing the address range

I chose **192.168.10.0/24**, from the 192.168.0.0/16 private block (RFC 1918). Private addresses are free to use inside any organisation and are never routed on the internet, which is why the router needs NAT (Part 3). A /24 gives 256 addresses, which is plenty for one floor.

### 2.2 Subnetting: wired and wireless separated

I split the /24 into smaller subnets so wired and wireless devices are in **separate subnets**. This means broadcasts from one don't reach the other, a laptop or phone (more likely to be lost or infected) can't reach the PCs and printer directly unless the router allows it, and each subnet can have its own rules and DHCP settings. Borrowing 2 bits from the host part gives /26 subnets of 64 addresses (62 usable). I used two of them, plus the upper half kept as a /25 for later.

| Subnet | Mask | Usable hosts | Broadcast | # | Gateway | Used for |
|---|---|---|---|---|---|---|
| **192.168.10.0/26** | 255.255.255.192 | 192.168.10.1 – 192.168.10.62 | 192.168.10.63 | 62 | 192.168.10.1 | PCs, printer, switches |
| **192.168.10.64/26** | 255.255.255.192 | 192.168.10.65 – 192.168.10.126 | 192.168.10.127 | 62 | 192.168.10.65 | Access point and Wi-Fi devices |
| **192.168.10.128/25** | 255.255.255.128 | 192.168.10.129 – 192.168.10.254 | 192.168.10.255 | 126 | — | Free for later (guest Wi-Fi, servers…) |
| **203.0.113.8/30** | 255.255.255.252 | 203.0.113.9 – 203.0.113.10 | 203.0.113.11 | 2 | 203.0.113.9 | Public link between R1 and the ISP |

**How /26 works:** the mask is `11111111.11111111.11111111.11000000` = 255.255.255.192. The first 26 bits are the network, and the last 6 bits ($2^6$ = 64 addresses) are for hosts. The first address of each block is the network address and the last is the broadcast address, so 64 – 2 = **62 usable** addresses per subnet. The blocks start every 64: .0, .64, .128 and .192. The WAN link is a /30 (4 addresses, 2 usable), the usual size for a link between two routers.

### 2.3 Address assignment

In each subnet, the first addresses are kept for devices that need a **fixed address**: the router (the gateway is always the first usable address), the switches' management interfaces, the printer and the access point. Everything else gets an address from **DHCP**. Wired: fixed 192.168.10.1 – 192.168.10.9, DHCP pool 192.168.10.10 – 192.168.10.62. Wireless: fixed 192.168.10.65 – 192.168.10.69, DHCP pool 192.168.10.70 – 192.168.10.126. The DHCP addresses shown are the ones the devices would typically receive.

| Device | Interface | IP address | Subnet mask | Default gateway | How |
|---|---|---|---|---|---|
| Router R1 | G0/0 (WAN, to ISP) | **203.0.113.10**/30 | 255.255.255.252 | — | Static (from ISP) |
| Router R1 | G0/1 (wired LAN) | **192.168.10.1**/26 | 255.255.255.192 | — | Static (gateway) |
| Router R1 | G0/2 (wireless LAN) | **192.168.10.65**/26 | 255.255.255.192 | — | Static (gateway) |
| Switch SW1 | Management | **192.168.10.2**/26 | 255.255.255.192 | 192.168.10.1 | Static |
| Switch SW2 | Management | **192.168.10.3**/26 | 255.255.255.192 | 192.168.10.1 | Static |
| Printer | Ethernet | **192.168.10.5**/26 | 255.255.255.192 | 192.168.10.1 | Static (so it never changes) |
| PC1 | Ethernet | **192.168.10.11**/26 | 255.255.255.192 | 192.168.10.1 | DHCP |
| PC2 | Ethernet | **192.168.10.12**/26 | 255.255.255.192 | 192.168.10.1 | DHCP |
| PC3 | Ethernet | **192.168.10.13**/26 | 255.255.255.192 | 192.168.10.1 | DHCP |
| PC4 | Ethernet | **192.168.10.14**/26 | 255.255.255.192 | 192.168.10.1 | DHCP |
| Access point AP1 | Ethernet (uplink + management) | **192.168.10.66**/26 | 255.255.255.192 | 192.168.10.65 | Static |
| Laptop (wireless 1) | Wi-Fi | **192.168.10.70**/26 | 255.255.255.192 | 192.168.10.65 | DHCP |
| Phone (wireless 2) | Wi-Fi | **192.168.10.71**/26 | 255.255.255.192 | 192.168.10.65 | DHCP |

The **default gateway** is the router interface in a device's own subnet: **192.168.10.1** for the wired devices and **192.168.10.65** for the wireless ones. A device sends any packet for another network (the other subnet or the internet) to its gateway. All devices use DNS servers 1.1.1.1 and 8.8.8.8. The printer has a fixed address so the PCs' printer settings never break. With DHCP, it could also get a *reservation* (always the same address for its MAC).

## 2.4 The role of DHCP

**DHCP** (Dynamic Host Configuration Protocol) gives devices their network settings automatically when they connect, so nobody has to type them on every PC and phone, and two devices can never be given the same address by mistake. Here the router R1 is the DHCP server, with one pool per subnet. Each pool hands out an **IP address**, the **subnet mask**, the **default gateway**, the **DNS servers** and a **lease time**. When the lease runs out, the address goes back to the pool, which suits Wi-Fi devices that come and go. When the laptop joins the Wi-Fi, it goes through the four **DORA** steps:

| # | Message | From → To | Content |
|---|---|---|---|
| 1 | **D**ISCOVER | 0.0.0.0 → 255.255.255.255 (broadcast)<br>MAC A4:83:E7:5B:00:21 → FF:FF:FF:FF:FF:FF | "I need an address." It has no IP yet, so it broadcasts (UDP 68 → 67). |
| 2 | **O**FFER | 192.168.10.65 → 192.168.10.70 | "You can have 192.168.10.70, mask 255.255.255.192, gateway 192.168.10.65, DNS 1.1.1.1, lease 2 h." |
| 3 | **R**EQUEST | 0.0.0.0 → 255.255.255.255 | "I accept 192.168.10.70." Broadcast, so any other DHCP server knows its offer was declined. |
| 4 | **A**CK | 192.168.10.65 → 192.168.10.70 | "Confirmed." The laptop now configures its interface. |

The configuration on the router, written as Cisco IOS commands:

```
ip dhcp excluded-address 192.168.10.1 192.168.10.9       ! fixed addresses are never handed out
ip dhcp excluded-address 192.168.10.65 192.168.10.69
ip dhcp pool WIRED
 network 192.168.10.0 255.255.255.192
 default-router 192.168.10.1
 dns-server 1.1.1.1 8.8.8.8
 lease 0 8                                              ! 8 hours: PCs stay all day
ip dhcp pool WIRELESS
 network 192.168.10.64 255.255.255.192
 default-router 192.168.10.65
 dns-server 1.1.1.1 8.8.8.8
 lease 0 2                                              ! 2 hours: phones come and go
```

The router can answer both subnets directly because it has an interface in each one. If the DHCP server were in another subnet, the router would need `ip helper-address` to forward the broadcast DISCOVER messages, because routers don't forward broadcasts.

## Part 3: Routing and Packet Forwarding

Scenario: the **laptop** (192.168.10.70) opens **https://www.example.com**. For the example I use 198.51.100.25 as the website's public IP address (an address reserved for documentation).

### 3.1 What happens, step by step

1. **Configuration:** the laptop already has its IP, mask, gateway and DNS from DHCP (Part 2).
2. **DNS lookup:** the laptop asks its DNS server (1.1.1.1, UDP port 53) for the IP of www.example.com and gets 198.51.100.25. The DNS query itself travels the same way as the steps below.
3. **Local or remote?** The laptop ANDs the destination with its mask: 198.51.100.25 AND 255.255.255.192 = **198.51.100.0**, which isn't its own network (192.168.10.64). So the packet must go to the **default gateway** 192.168.10.65. (For the AP at 192.168.10.66, the result would be 192.168.10.64, the same network, and it would send directly.)
4. **ARP for the gateway:** the laptop needs the gateway's MAC, so it ARPs for 192.168.10.65 and learns `00:1A:2B:3C:4D:03`. Note that the IP destination is the website, but the frame's destination MAC is the router.
5. **Building the packet:** TCP (for HTTPS, port 443) from a random source port 51514. The IP header has source 192.168.10.70, destination 198.51.100.25, TTL 64 and protocol 6. (The very first packets are the TCP handshake SYN, SYN-ACK and ACK, then the TLS handshake. They all follow this same path.)
6. **Over the air to the AP:** the packet goes in an **802.11 frame**. Wi-Fi frames have 3 addresses here: receiver = the AP's BSSID `00:24:6C:AA:00:02`, transmitter = the laptop `A4:83:E7:5B:00:21`, final destination = the router `00:1A:2B:3C:4D:03`. CSMA/CA decides when it may transmit, and the AP ACKs it.
7. **The AP bridges it to Ethernet:** the AP works at layer 2. It turns the 802.11 frame into an Ethernet frame (destination 00:1A:2B:3C:4D:03, source A4:83:E7:5B:00:21) and sends it up its cable to router port G0/2.
8. **The router processes the IP header** (3.2): it checks the frame's FCS, sees its own MAC, removes the Ethernet header, checks the IP header checksum, **decreases the TTL** (64 → 63), and looks up the destination in its **routing table** (3.3). The result is the default route, out G0/0.
9. **NAT** (3.4): because the source is a private address, the router replaces `192.168.10.70:51514` with its public address `203.0.113.10:40001`, remembers the translation, and recalculates the checksums.
10. **New frame for the next link:** the router ARPs for the ISP gateway 203.0.113.9 and puts the packet in a **new Ethernet frame**: source 00:1A:2B:3C:4D:01, destination 00:50:56:EE:00:09.
11. **Across the internet:** every router on the way does the same thing: reads the destination IP, decreases TTL, does a longest-prefix-match lookup (internet routers learn routes with protocols like BGP and OSPF), and builds a new frame for the next link. **The MAC addresses change on every hop, but the IP addresses don't** (after NAT). If TTL ever reaches 0, the packet is dropped and an ICMP "time exceeded" message is sent back, which prevents endless loops.
12. **The way back:** the server replies to `203.0.113.10:40001`. R1 finds that port in its NAT table, changes the destination back to `192.168.10.70:51514`, sees in its routing table that 192.168.10.64/26 is directly connected on G0/2, and sends it in a frame to the laptop's MAC. The AP converts it to an 802.11 frame, and the laptop's browser receives the web page.

### Headers on each part of the path

| Hop | Link | Layer 2: source → destination MAC | Source IP:port | Destination IP:port | TTL |
|---|---|---|---|---|---|
| 1 | Laptop → AP1 (802.11 Wi-Fi) | A4:83:E7:5B:00:21 → 00:24:6C:AA:00:02 (BSSID)<br>3rd address = router 00:1A:2B:3C:4D:03 | 192.168.10.70:51514 | 198.51.100.25:443 | 64 |
| 2 | AP1 → R1 G0/2 (Ethernet) | A4:83:E7:5B:00:21 → 00:1A:2B:3C:4D:03 | 192.168.10.70:51514 | 198.51.100.25:443 | 64 |
| 3 | R1 G0/0 → ISP gateway | 00:1A:2B:3C:4D:01 → 00:50:56:EE:00:09 | **203.0.113.10:40001** (NAT) | 198.51.100.25:443 | **63** |
| 4…n | Internet routers | new MACs on every link | 203.0.113.10:40001 | 198.51.100.25:443 | 62, 61, … |
| back | Reply arrives at R1 G0/0 | ISP MAC → 00:1A:2B:3C:4D:01 | 198.51.100.25:443 | 203.0.113.10:40001 | ≈ 50 |
| back | R1 G0/2 → AP1 → Laptop | 00:1A:2B:3C:4D:03 → A4:83:E7:5B:00:21 | 198.51.100.25:443 | **192.168.10.70:51514** (NAT reversed) | ≈ 49 |

## 3.2 How routers use the IP header

Switches only look at the MAC addresses in the frame. A router removes the frame and works on the **IPv4 header**:

| Field | What the router does with it |
|---|---|
| **Destination IP address** | The most important field: it's looked up in the routing table to choose the next hop and the outgoing interface. |
| **TTL (Time To Live)** | Decreased by 1 at every router. At 0 the packet is dropped and an ICMP "Time Exceeded" is sent back, so packets can't loop forever. |
| **Header checksum** | Checked when the packet arrives (damaged headers are dropped) and recalculated after TTL or NAT changes. |
| **Source IP address** | Normally not changed. Here, NAT replaces it at R1. It's also used by ACLs (firewall rules) and for sending ICMP errors. |
| **Total length, ID, flags, fragment offset** | If the next link's MTU is smaller than the packet, the router can fragment it (unless the "Don't Fragment" flag is set, in which case it drops the packet and tells the sender). |
| **Protocol** (6 = TCP), **DSCP** | Protocol tells NAT and firewalls which header comes next (here TCP, for the ports). DSCP can be used to prioritise traffic (QoS). |

## 3.3 How the router chooses the next hop

R1's routing table. **C** routes are networks it's directly connected to, and **S*** is the static **default route** to the ISP ("gateway of last resort"):

| Code | Destination | Next hop | Interface |
|---|---|---|---|
| C | 192.168.10.0/26 | directly connected | G0/1 |
| C | 192.168.10.64/26 | directly connected | G0/2 |
| C | 203.0.113.8/30 | directly connected | G0/0 |
| S* | 0.0.0.0/0 | 203.0.113.9 | G0/0 |

The router compares the destination IP with every route and picks the matching route with the **longest prefix** (the most specific one). The default route 0.0.0.0/0 matches everything, but it has the shortest prefix, so it's only used when nothing more specific matches.

**Outgoing packet to 198.51.100.25**

| Route | Match? | Prefix |
|---|---|---|
| 192.168.10.0/26 | ✘ no match | /26 |
| 192.168.10.64/26 | ✘ no match | /26 |
| 203.0.113.8/30 | ✘ no match | /30 |
| **0.0.0.0/0** | **✔ matches** | **/0** |

→ only the default route matches: **next hop 203.0.113.9 via G0/0**.

**Reply to 192.168.10.70 (after NAT)**

| Route | Match? | Prefix |
|---|---|---|
| 192.168.10.0/26 | ✘ no match | /26 |
| **192.168.10.64/26** | **✔ matches** | **/26** |
| 203.0.113.8/30 | ✘ no match | /30 |
| **0.0.0.0/0** | **✔ matches** | **/0** |

→ /26 beats /0: **directly connected, out G0/2**, to the laptop itself.

When the next hop is "directly connected", the router delivers to the destination host itself and ARPs for *its* MAC. Otherwise it ARPs for the next-hop router's MAC. Either way, forwarding never changes the destination IP in the packet (only NAT does, at the edge of the network). Only the frame around it changes.

### 3.4 The role of NAT at the router

The 192.168.10.x addresses are private, so the internet can't route replies to them, and the ISP gives the company only **one public address** (203.0.113.10). **NAT** (Network Address Translation) on R1 solves both problems. More precisely, it's **PAT** (Port Address Translation, "NAT overload"): all 6 PCs and wireless devices share the one public IP, and the router tells their connections apart by **port number**. It keeps a translation table:

| Inside local (private) | Inside global (public) | Outside (website) | Device |
|---|---|---|---|
| 192.168.10.70:51514 | 203.0.113.10:40001 | 198.51.100.25:443 | Laptop |
| 192.168.10.71:60200 | 203.0.113.10:40002 | 198.51.100.25:443 | Phone |
| 192.168.10.11:51514 | 203.0.113.10:40003 | 198.51.100.25:443 | PC1 (same source port as the laptop!) |

The last row shows why ports are translated too: PC1 and the laptop happen to use the same source port, so the router gives each connection a different public port to keep the replies apart. For outgoing packets, NAT rewrites the **source** IP and port. For replies, it looks up the destination port and rewrites the **destination** back. It also protects the LAN, because a packet from the internet that doesn't match an existing entry has nowhere to go, so outside devices can't start connections to the PCs. The router configuration:

```
interface G0/0
 ip address 203.0.113.10 255.255.255.252
 ip nat outside
interface G0/1
 ip address 192.168.10.1 255.255.255.192
 ip nat inside
interface G0/2
 ip address 192.168.10.65 255.255.255.192
 ip nat inside
access-list 1 permit 192.168.10.0 0.0.0.127        ! both LAN subnets (.0 – .127)
ip nat inside source list 1 interface G0/0 overload
ip route 0.0.0.0 0.0.0.0 203.0.113.9            ! default route to the ISP
```

**Summary of the path:** laptop → (802.11, CSMA/CA) → AP → (Ethernet) → R1 G0/2 → routing table: default route → NAT 192.168.10.70:51514 → 203.0.113.10:40001 → G0/0 → ISP → internet routers (TTL – 1 and a new frame at each hop) → 198.51.100.25. The reply comes back to the public address, NAT translates it back to the laptop's private address, and R1 delivers it through the AP. At layer 2 the addresses change on every link, at layer 3 they stay the same end to end, except where NAT rewrites them at the edge.